ROUTE MAP · USE ↑ / ↓ WITHIN CHAPTERS

One journey, six checkpoints

01 Origins → 02 REST + HTTP → 03 Toolchain → 04 PHP build → 05 Clients + ops → 06 AI copilot

Format

Concept → worked example → decision → practice. Vertical slides contain optional depth.

Controls

S speaker view · F fullscreen · 0 overview · `Ctrl/⌘ + K` search

A COMPRESSED HISTORY

From procedures to resources

1969
ARPANET demonstrates packet-switched networking.

1989–91
Berners-Lee proposes the Web: URI, HTTP, HTML.

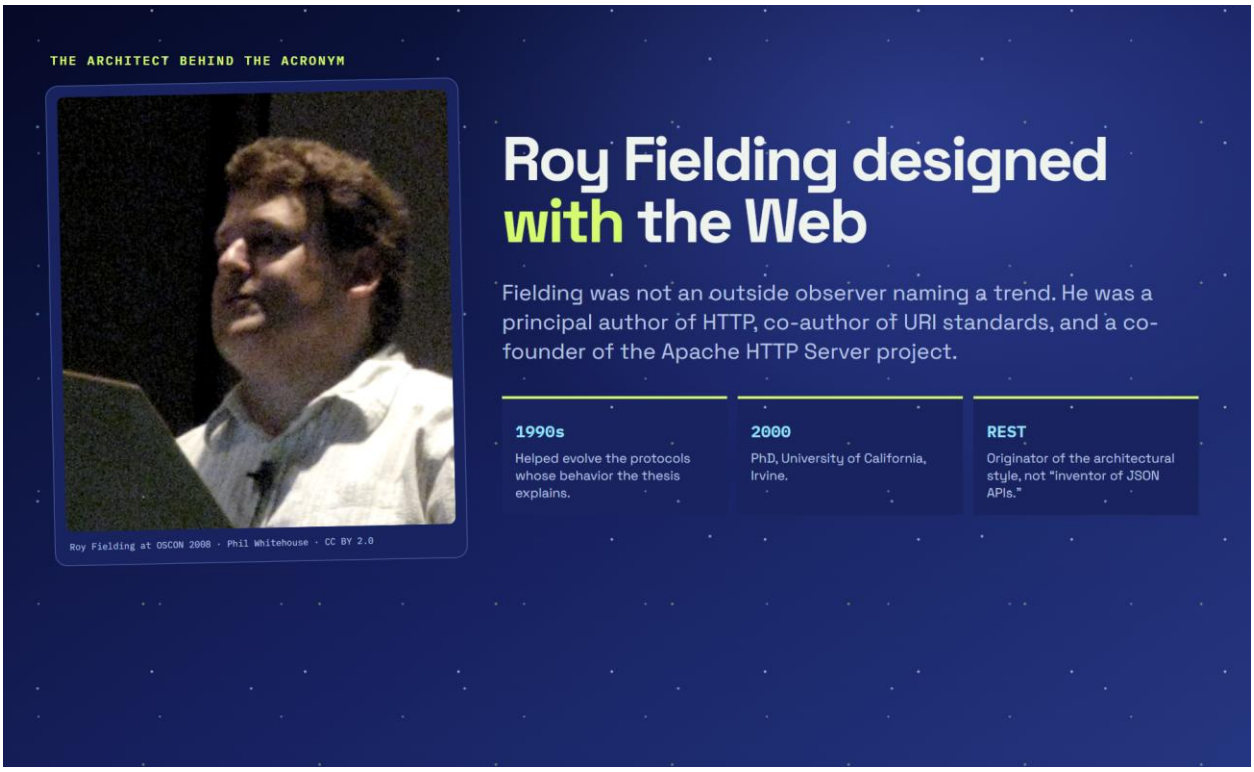
1996–99
HTTP/1.0 then 1.1 standardize web semantics.

2000
Roy Fielding names REST in his dissertation.

2000s
SOAP/WSDL and "Web 2.0" APIs grow in parallel.

2010s→
Cloud, mobile, microservices, OpenAPI, GraphQL, gRPC.

> REST did not invent HTTP. It described the architectural constraints that made the Web evolvable at global scale.



https://youtu.be/w5j2KwzzB-0

The dissertation · 2000

Architectural Styles and the Design of Network-based Software Architectures

The question

How should we understand and design software architectures for distributed hypermedia systems at Internet scale?

The contribution

A framework for evaluating network architectures, then REST as a derived style tailored to the Web's requirements.

REST occupies Chapter 5. The thesis is broader: architecture, properties, styles, Web requirements, derivation, and evaluation.

How the argument works

REST was derived, not brainstormed

ØNull styleNo constraints

+

C/S Client–serverSeparation

+

$0StatelessVisibility + scale

+

TTLCacheEfficiency

+

UIUniform interfaceIndependent evolution

+

LLayersIntermediaries

Each constraint trades freedom for system properties. Statelessness improves visibility and scalability, but can increase per-request data and network cost.

What Fielding did not say

“REST means HTTP + JSON”

JSON is absent from the definition. REST is a coordinated set of architectural constraints; HTTP is one protocol designed to support them.

“A neat URL is RESTful”

Resource naming alone does not provide statelessness, cache semantics, self-descriptive messages, or hypermedia.

“REST is remote database CRUD”

A resource is a conceptual mapping whose representation can change. It need not correspond to a table.

“Every API should be REST”

The style optimizes particular properties. Different constraints can better fit streaming, low-latency RPC, or client-shaped graph queries.

RPC asks for actions.
REST transfers representations.

Procedure mindset

POST /getBook

POST /createBook

POST /deleteBook

The method is hidden in the path. Infrastructure sees every operation as POST.

Resource mindset

GET /books/42

POST /books

DELETE /books/42

Uniform HTTP semantics make intent visible to clients, caches, gateways, and observability tools.

Choose an interaction model, not a tribe

Style	Optimizes for	Strong fit
REST/HTTP	Universality, evolvability	Public APIs, CRUD-ish doma
GraphQL	Client-shaped queries	Complex UI data graphs
gRPC	Typed, efficient RPC	Internal service-to-service
Webhooks	Server-pushed events	Asynchronous notifications
WebSocket/SSE	Streaming updates	Realtime feeds and collabora

REST is a set of constraints,
HTTP is the vocabulary

A “RESTful” API is not JSON over HTTP. Its behavior emerges from architectural constraints and protocol semantics.

Fielding’s constraints

The architecture of universality

Client–server

Separate user interface concerns from data and domain concerns.

Stateless

Each request carries all context required to understand it.

Cacheable

Responses declare whether and how intermediaries may reuse them.

Uniform interface

Resources, representations, self-descriptive messages, hypermedia.

Layered system

A client need not know whether it speaks to an origin, proxy, or gateway.

Code on demand

Optional: servers may extend clients with executable code.

The hardest constraint does the most work

URI

Identify resources

/orders/817 identifies a conceptual resource, not a table row.

REP

Manipulate via representations

JSON is one representation. The resource is not the JSON document itself.

MSG

Self-descriptive messages

Method, status, headers, media type, and body explain processing.

HATEOAS

Hypermedia

Responses expose valid next actions, reducing out-of-band coupling.

Most “REST APIs” implement the first three partially and little hypermedia. Name the compromise honestly.

Nouns in paths, semantics in methods

GET/bookscollection

POST/bookscreate

GET/books/42member

PATCH/books/42partial update

DELETE/books/42remove

Model domain concepts

POST /orders/817/cancellation can be clearer than POST /cancelOrder: cancellation is a resource with state, authorization, time, and audit history.

Avoid mirroring database schema blindly. Design the API around consumer workflows.

Safe ≠ idempotent

Method	Safe?	Idempotent?	T
GET / HEAD	Yes	Yes	R
POST	No	No*	C
PUT	No	Yes	R
PATCH	No	Depends	A
DELETE	No	Yes	E

*POST can be made retry-safe using an Idempotency-Key, common in payment APIs.

Communicate outcomes precisely

2xx Success

200 read/update
201 created + Location
202 accepted async
204 no body

3xx Redirection

304 use cached representation
307/308 preserve method

4xx Client

400 malformed
401 unauthenticated
403 forbidden
404/409/422/429

5xx Server

500 unexpected
502 bad upstream
503 unavailable
504 upstream timeout

Predictable envelopes, actionable errors

```
{
  "data": [{ "id": 42, "title": "RESTful Web APIs" }],
  "meta": { "page": 1, "per_page": 20, "total": 81 },
  "links": {
    "self": "/books?page=1",
    "next": "/books?page=2"
  }
}
{
  "type": "https://api.example.test/problems/validation",
  "title": "Request validation failed",
```

```
 "status": 422,
 "detail": "Two fields require attention.",
 "errors": {
  "title": ["This field is required."]
 }
}
```


RFC 9457 “Problem Details” gives errors a standard shape. Never leak stack traces or internal SQL.

Caching is correctness with a clock

Response

HTTP/1.1 200 OK

Cache-Control: public, max-age=60

ETag: "books-v7"

→
←

Revalidation

GET /books

If-None-Match: "books-v7"

HTTP/1.1 304 Not Modified

Caching reduces latency, compute, database load, and carbon. Incorrect caching can expose one user’s data to another. Set cache policy intentionally.

The API ecosystem:
service, client, contract, infrastructure

APIs are developed by a toolchain and operated as a socio-technical system, not merely coded in a framework.

Tools by job-to-be-done

Frameworks

Laravel, Symfony, Slim, API Platform, Laminas; also Express, FastAPI, Spring, ASP.NET.

Data

PostgreSQL, MySQL, Redis, Doctrine ORM, Eloquent, migrations, queues.

Quality

PHPUnit/Pest, PHPStan/Psalm, PHP-CS-Fixer, Rector, mutation and contract tests.

Contract

OpenAPI, JSON Schema, Swagger UI, Redoc, Spectral, generated SDKs.

Composer Docker GitHub Actions Sentry Open Telemetry Vault

Inspect, automate, integrate

Human exploration

curl / HTTPie for reproducible commands. Bruno, Postman, Insomnia for collections, environments, and collaboration.

Application clients

Browser fetch, Axios, PHP Guzzle/Symfony HttpClient, mobile SDKs, generated clients.

Verification

Pact for consumer contracts, Newman/Bruno CLI for collections, k6 for load, OWASP ZAP for dynamic security.

A copied GUI request is not documentation. A checked-in contract and executable test are.

Production adds layers on purpose

DNSRoute name

CDN/WAF Cache + defend

GatewayAuth + limits

ServiceDomain logic

DataState

TelemetryObserve

Compute

VMs, containers, serverless, Kubernetes. Start with the least operational complexity that meets constraints.

Delivery

CI checks → immutable artifact → migration strategy → gradual rollout → rollback.

Reliability

Timeouts, retries with jitter, circuit breakers, queues, health probes, backups, disaster recovery.

Build a Book API with modern PHP

A step-by-step Laravel implementation, with framework-neutral principles called out along the way.

Design from consumer journeys

Stories

- Browse books with pagination and search
- Read one book
- Create and edit as an authenticated librarian
- Delete while preserving audit history

Initial contract

GET /api/v1/books?search=&page=

POST /api/v1/books

GET /api/v1/books/{book}

PATCH /api/v1/books/{book}

DELETE /api/v1/books/{book}

Define examples, status codes, validation, authorization, pagination, and error shape before controller code.

Create and inspect the service

```
composer create-project laravel/laravel book-api
cd book-api
php artisan install:api
php artisan make:model Book -mf
php artisan make:controller Api/V1/BookController --api
php artisan make:request StoreBookRequest
php artisan make:request UpdateBookRequest
php artisan make:resource BookResource
```

Model

Domain data and persistence behavior.

Request

Input authorization and validation boundary.

Resource

Stable output representation boundary.

Migration + model

Migration

```
Schema::create('books', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('author');
    $table->string('isbn', 13)->unique();
    $table->unsignedSmallInteger('published_year')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

Model

```
final class Book extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'title', 'author', 'isbn', 'published_year',
    ];
}
```

Factory

```
public function definition(): array
{
    return [
        'title' => fake()->sentence(3),
        'author' => fake()->name(),
        'isbn' => fake()->unique()->isbn13(),
        'published_year' => fake()->numberBetween(1900, date('Y')),
    ];
```

Version at the boundary

```
// routes/api.php
Route::prefix('v1')
    ->name('api.v1.')
    ->group(function (): void {
        Route::apiResource('books', BookController::class)
            ->only(['index', 'show']);
```

```
    Route::middleware('auth:sanctum')->group(function (): void {
        Route::apiResource('books', BookController::class)
            ->only(['store', 'update', 'destroy']);
    });
});
```

Versioning is a compatibility policy, not just /v1. Prefer additive change; version only when behavior must break.

Reject ambiguity before domain logic

```
final class StoreBookRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()?->can('create', Book::class) ?? false;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'author' => ['required', 'string', 'max:255'],
            'isbn' => ['required', 'digits:13', 'unique:books,isbn'],
            'published_year' => ['nullable', 'integer', 'between:1450,'.date('Y')],
        ];
    }
```

}

Validation proves shape, not business truth. “ISBN exists” and “user may publish this title” may require domain services or policies.

Never serialize the database accidentally

```
final class BookResource extends JsonResource
{
  public function toArray(Request $request): array
  {
    return [
      'id' => $this->id,
      'title' => $this->title,
      'author' => $this->author,
      'isbn' => $this->isbn,
      'published_year' => $this->published_year,
      'links' => [
        'self' => route('api.v1.books.show', $this->resource),
      ],
    ];
  }
}
```

A resource/transformer prevents internal columns, renamed fields, and ORM relationships from becoming accidental public contracts.

Filter, order, paginate, cap

```
public function index(Request $request): AnonymousResourceCollection
{
  $validated = $request->validate([
```

```
        'search' => ['nullable', 'string', 'max:100'],
        'per_page' => ['nullable', 'integer', 'between:1,100'],
    ]);

    $books = Book::query()
        ->when($validated['search'] ?? null, fn ($query, $search) =>
            $query->where(fn ($query) => $query
                ->where('title', 'like', "%{$search}%")
                ->orWhere('author', 'like', "%{$search}%")))
        ->orderByDesc('id')
        ->paginate($validated['per_page'] ?? 20)
        ->withQueryString();

    return BookResource::collection($books);
}
```

201 + representation + Location

```
public function store(StoreBookRequest $request): JsonResponse
{
    $book = Book::create($request->validated());

    return (new BookResource($book))
        ->response()
        ->setStatusCode(Response::HTTP_CREATED)
        ->header(
            'Location',
            route('api.v1.books.show', $book)
```

```
    );
}
```

Thin controller: transport orchestration belongs here. Complex workflows belong in application/domain services.

Transactions: wrap multiple state changes atomically with DB::transaction().

Let route binding carry identity

```
public function show(Book $book): BookResource
{
  return new BookResource($book);
}

public function update(UpdateBookRequest $request, Book $book): BookResource
{
  $book->update($request->validated());

  return new BookResource($book->refresh());
}

public function destroy(Book $book): Response
{
  $this->authorize('delete', $book);
  $book->delete();

  return response()->noContent();
}
```

Missing route-bound models become 404. A successful delete returns 204; repeated DELETE may return 404 while still being idempotent in effect.

Identity is not permission

Authenticate

Who is calling? Sessions for same-origin apps, Laravel Sanctum tokens, OAuth 2.1/OIDC for delegated access.

Authorize

May this principal perform this action on this resource? Use policies, roles, scopes, ownership.

Audit

Who did what, when, from where, to which object? Keep security logs structured and tamper-resistant.

Authorization: Bearer <opaque-or-JWT-access-token>

Never place secrets in URLs. Hash stored API tokens. Rotate keys. Keep token lifetimes and scopes narrow.

One exception pipeline, no leaked internals

```
// bootstrap/app.php (Laravel 11+)
->withExceptions(function (Exceptions $exceptions): void {
  $exceptions->shouldRenderJsonWhen(
    fn (Request $request, Throwable $e) =>
      $request->is('api/*') || $request->expectsJson()
  );

  $exceptions->respond(function (Response $response) {
    $response->headers->set('X-Request-Id', request()->header('X-Request-Id', Str::uuid()));
    return $response;
  });
})
```

Map known domain failures to stable problem types. Log detailed exceptions server-side with request IDs; return safe details client-side.

Feature test the public contract

```
it('creates a book for an authorized librarian', function () {
  $user = User::factory()->librarian()->create();

  $response = $this->actingAs($user)->postJson('/api/v1/books', [
    'title' => 'HTTP Semantics',
    'author' => 'A. Engineer',
    'isbn' => '9781234567890',
    'published_year' => 2026,
  ]);

  $response
    ->assertCreated()
    ->assertHeader('Location')
    ->assertJsonPath('data.title', 'HTTP Semantics');

  $this->assertDatabaseHas('books', ['isbn' => '9781234567890']);
});
```

Also test: unauthenticated 401, forbidden 403, invalid 422, missing 404, duplicate ISBN, pagination limits, and unexpected exceptions.

OpenAPI makes behavior portable

```
openapi: 3.1.0
info: { title: Book API, version: 1.0.0 }
paths:
 /api/v1/books/{book}:
```

```
  get:
    operationId: getBook
    parameters:
      - in: path
        name: book
        required: true
        schema: { type: integer, minimum: 1 }
    responses:
      '200':
        description: Book found
        content:
          application/json:
            schema: { $ref: '#/components/schemas/BookResponse' }
      '404': { $ref: '#/components/responses/NotFound' }
```

Lint it, publish it, generate examples/SDKs, and detect breaking changes in CI. The implementation and contract must be tested against each other.

PHP gives you a spectrum

Tool	Character	Choose
Laravel	Integrated, expressive ecosystem	Product
Symfony	Modular, explicit, enterprise-ready	You nee
API Platform	Contract-first automation on Symfony	Resourc
Slim	Minimal PSR middleware framework	Small se

Tool	Character	Choose
Plain PHP + PSR	Maximum control, minimum magic	Special

Consume defensively.
Operate observably.

A successful response is only one branch. Real clients handle latency, cancellation, authentication, retries, partial failure, and change.

Start with curl

```
curl --fail-with-body \
  --request POST 'https://api.example.test/api/v1/books' \
  --header 'Accept: application/json' \
  --header 'Content-Type: application/json' \
  --header 'Authorization: Bearer '$API_TOKEN \
  --data '{"title":"HTTP Semantics","author":"A. Engineer","isbn":"9781234567890"}'
```

Why CLI first?

Portable, scriptable, diffable, and close to the wire. Use -v to inspect negotiation and TLS.

Secret hygiene

Read tokens from environment or a secret store. Shell history, screenshots, and committed collections leak credentials.

Fetch with timeout and typed failure paths

```
async function getBook(id, signal = AbortSignal.timeout(5000)) {
  const response = await fetch('/api/v1/books/' + encodeURIComponent(id), {
    headers: { Accept: 'application/json' },
    credentials: 'same-origin',
    signal,
  });
```

```
  const body = response.status === 204 ? null : await response.json();


  if (!response.ok) {
    throw new ApiError(response.status, body?.title, body);
  }


  return body.data;
}
```

fetch rejects on network failure, not on HTTP 404/500. Check response.ok. Cancellation is a feature, not cleanup

Consume services server-to-server

```
$response = Http::baseUrl(config('services.books.url'))
    ->acceptJson()
    ->withToken(config('services.books.token'))
    ->timeout(5)
    ->connectTimeout(2)
    ->retry(3, 200, throw: false)
    ->get("/api/v1/books/{$id}");


if ($response->notFound()) {
    return null;
}


$response->throw();
return BookData::from($response->json('data'));
```

Retry only transient failures and safe/idempotent operations. Bound total retry time. Never create a retry storm against a struggling dependency.

CORS is not authentication

Cross-origin policy

The browser asks whether frontend origin A may read responses from API origin B. Configure exact allowed origins, methods, and headers.

Access-Control-Allow-Origin: https://app.example.com

Vary: Origin

Credential policy

Cookie auth requires CSRF defenses and careful SameSite settings. Bearer tokens avoid CSRF but create theft/storage risks.

Never use * with credentials. Non-browser clients ignore CORS entirely.

Threat-model the object, not only the endpoint

- Object-level authorization on every ID
- Property-level allowlists for input/output
- Rate limits by identity and cost
- Strict validation and body size limits
- Parameterized database access
- TLS everywhere; secure headers
- Inventory and retire old API versions
- Protect SSRF-capable URL fetches
- Do not trust JWT claims without verification
- Redact tokens and PII from logs

Use the OWASP API Security Top 10 as a baseline, then threat-model your domain's assets and abuse cases.

Can you explain one bad request?

Metrics

Rate, errors, duration, saturation. Break down by route template, status class, dependency; avoid high-cardinality user IDs.

Logs

Structured events with request/trace IDs, sanitized context, outcome, timing. Logs should answer who/what/when.

Traces

Propagate context across gateway, PHP service, queue, database, and downstream APIs with OpenTelemetry.

Availability is a user experience. Define SLOs, alert on symptoms, and keep enough telemetry to debug without reproducing production.

From commit to production

LintStyle + spec

AnalyzePHPStan

TestUnit + feature

ScanDeps + image

DeployCanary

VerifySLO + rollback

Database changes: expand → deploy compatible code → migrate data → contract. Avoid a single destructive migration.

Runtime: PHP-FPM behind Nginx/Caddy, or long-running workers via FrankenPHP/RoadRunner/Octane. Understand worker state before optimizing.

Evolution beats version proliferation

Usually safe

- Add optional response fields
- Add optional request fields
- Add endpoints or enum values*
- Improve performance without semantic change

Usually breaking

- Remove or rename fields
- Change types or meaning
- Make optional input required
- Change status/error behavior

*Clients that model enums exhaustively can break. Compatibility depends on documented client expectations, not only JSON validity.

AI can accelerate the loop.
It cannot own the contract.

Use models for bounded transformation, critique, test generation, and integration assistance. Keep architecture, security, and verification human-owned.

Where AI earns its place

Discover

Turn user stories into candidate resources, edge cases, threat questions, and an initial OpenAPI draft.

Implement

Scaffold requests/resources/tests, explain framework APIs, migrate repetitive code, produce fixtures.

Integrate

Generate typed clients from contracts, map error behavior, diagnose HTTP traces, draft usage examples.

Review

Compare code to OpenAPI, search for missing authorization, inconsistent status codes, N+1 queries.

Test

Generate boundary, property, mutation, contract, and adversarial cases from explicit invariants.

Operate

Summarize sanitized logs/traces, draft incident timelines, cluster errors, propose runbook steps.

Context + constraints + acceptance tests

ROLE: Senior PHP API engineer reviewing a Laravel 12 service.

CONTEXT: OpenAPI excerpt, route, request, resource, controller, policy.

TASK: Implement POST /api/v1/books.

CONSTRAINTS:

- PHP 8.4, strict types, existing project conventions

- 201 with Location; RFC 9457 validation errors

- authorization through BookPolicy; no new dependency

- never mass-assign unvalidated input

ACCEPTANCE:

- feature tests for 201, 401, 403, 422, duplicate ISBN

- PHPStan level 8 passes

OUTPUT:

- minimal patch, then assumptions and commands to verify

Give the model authoritative local context. Ask it to state uncertainty. Prefer small patches you can review over entire generated applications.

Let the spec anchor human + machine

IntentUser journey

ContractOpenAPI

AI draft Code + tests

Human review Domain + threat

CI proof Static + dynamic

ObserveReal behavior

For service code

Ask for a patch against existing conventions, test first where possible, then run formatter, static analysis, tests, and API contract checks.

For integration

Generate clients from OpenAPI, then ask AI to wrap them with application-specific error mapping, retries, telemetry, and cancellation.

What is wrong with this AI-generated endpoint?

```
public function show(Request $request): JsonResponse
{
   $book = Book::find($request->id);
   return response()->json($book);
}
```

No route-bound ID or input validation; missing book returns 200 null; possible object-level authorization gap; raw model leaks columns and creates contract coupling; return type/representation behavior is accidental; no cache or request context policy.

Never delegate trust

Do not send

Production secrets, access tokens, raw customer PII, proprietary data without approval, unredacted incident logs.

Do not accept blindly

Invented framework APIs, insecure defaults, dependency additions, legal claims, migrations, authorization logic, benchmark claims.

Constrain tools

Least-privilege credentials, sandbox, allowlisted commands, reviewed diffs, protected branches, audit logs, cost limits.

Verify outputs

Read the diff. Run tests and analyzers. Compare to official docs. Exercise unhappy paths. Threat-model the result.

A library kiosk network

Constraints

1,000 kiosks; intermittent connectivity; personal loan history; inventory changes; payment provider; peak at school closing time.

Architecture decisions

Resource model · offline/idempotency strategy · auth · caching · sync conflicts · async work · observability · API evolution.

The instructor traces request and failure paths, showing how each system constraint changes the design.

Which statement best captures REST?

REST means JSON over HTTPREST is remote CRUDREST is a constrained architecture for networked systemsREST replaces every other API style

Correct. Constraints produce properties such as scalability, visibility, and evolvability.

Before you call it production-ready

Contract

Consumer workflow, resource model, methods, statuses, errors, examples, evolution policy.

Code

Validated input, explicit output, authorization, transactions, pagination, bounded queries.

Proof

Static analysis, feature/contract/security/load tests, reviewed migration and rollback.

Operations

Secrets, rate limits, telemetry, SLOs, backups, runbooks, ownership, deprecation plan.

Design the contract. Respect HTTP. Make failure explicit. Verify every generated claim. Observe what users actually experience.

Primary references

Standards + architecture

Fielding dissertation · RFC 9110 HTTP Semantics · RFC 9457 Problem Details · RFC 9745 Deprecation · OpenAPI 3.1 · JSON Schema.

Practice + security

Laravel and Symfony docs · OWASP API Security Top 10 · OAuth 2.1 drafts/OIDC · OpenTelemetry semantic conventions · Pact contract testing.

The best API feels unsurprising to use, difficult to misuse, observable when it fails, and inexpensive to evolve.